Why Build a Computer Network?

- Before computer networks, people sent and received information by hand, using the postal service. This is slow and can be unreliable.
- Computer networks enable faster, more efficient modes of communication, i.e., email, video conferencing, etc.
- Computer networks and the sharing of electronic data encourage the use of standard policies and procedures.
- Computer networks provide backup and recovery support for our data, i.e., redundancy.
- Computer networks lead to cost savings.

Some Basic Networking Rules

- The computers in a network must use the same procedures for sending and receiving data. We call these ***communication protocols***.
- Data must be delivered uncorrupted. If it is corrupted, it's useless. (There are Exceptions)
- Computers in a network must be capable of determining the ***origin*** and ***destination*** of a piece of information, i.e., its **IP** and ***Mac Address***.

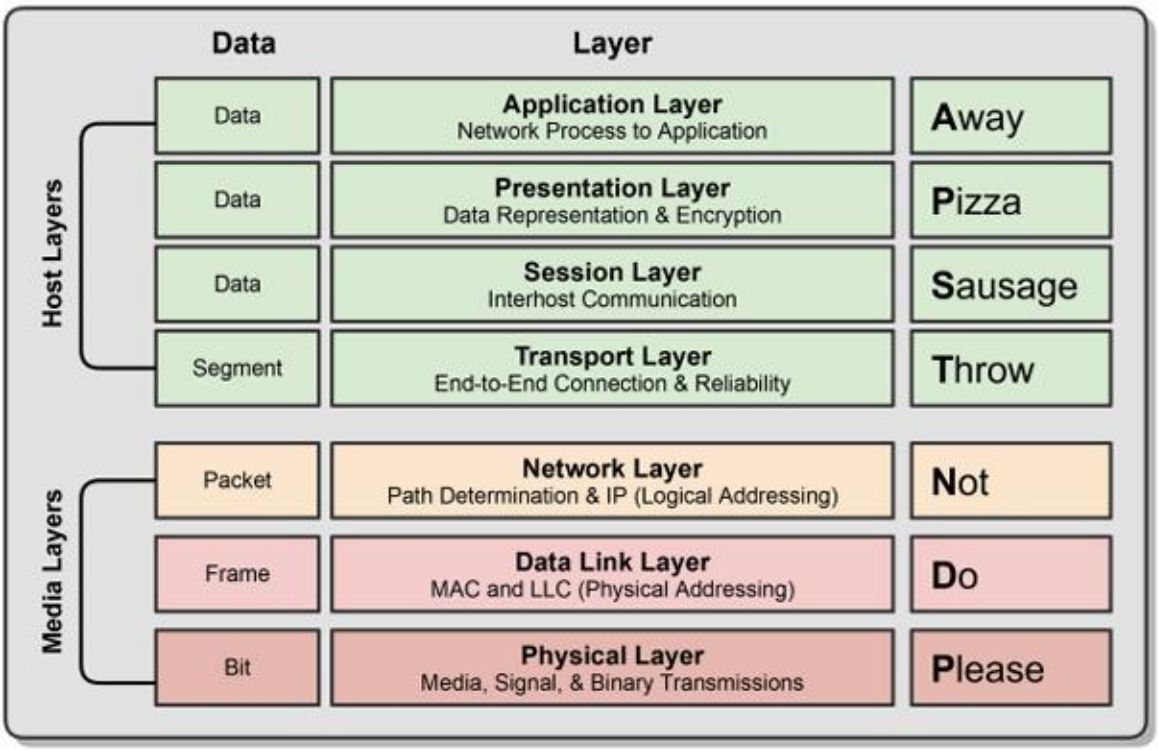
What is a Network?



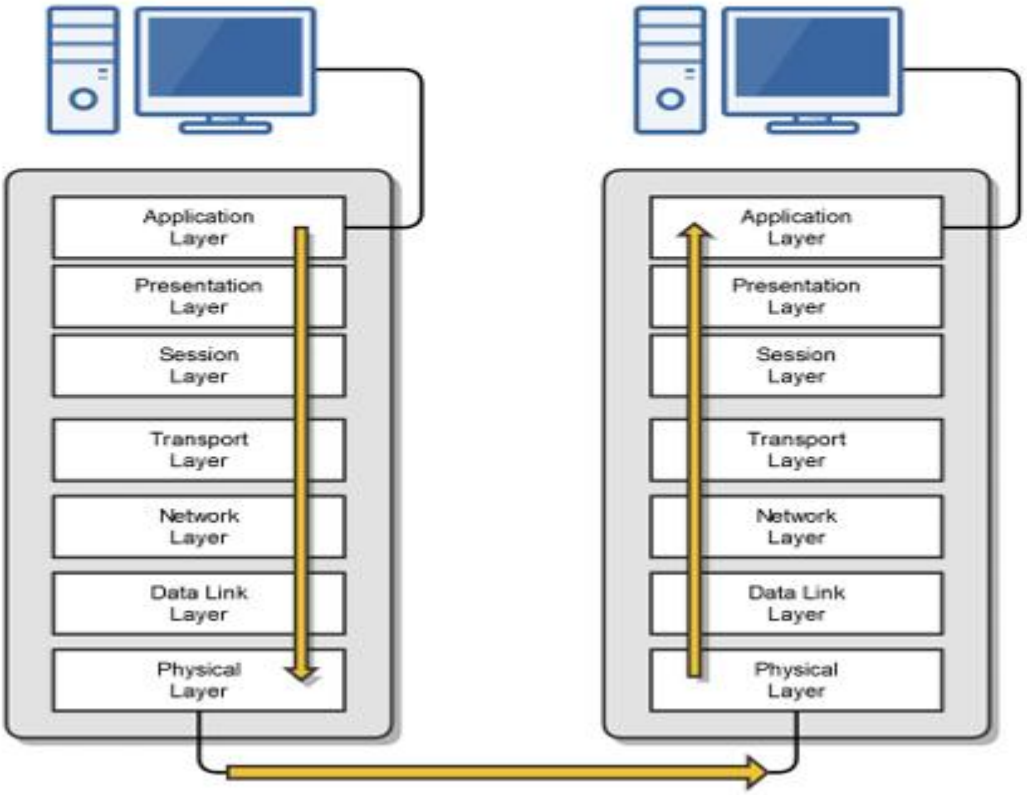
- In its simplest form, a network is nothing more than “two connected computers sharing resources with one another.”

- It is composed of two main aspects:
  - **Physical Connection** (wires, cables, wireless media)
  - **Logical Connection** (data transporting across the physical media)

The OSI Model Visualized

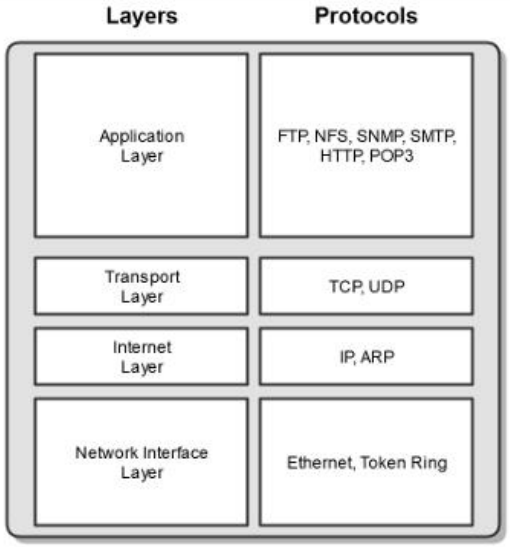


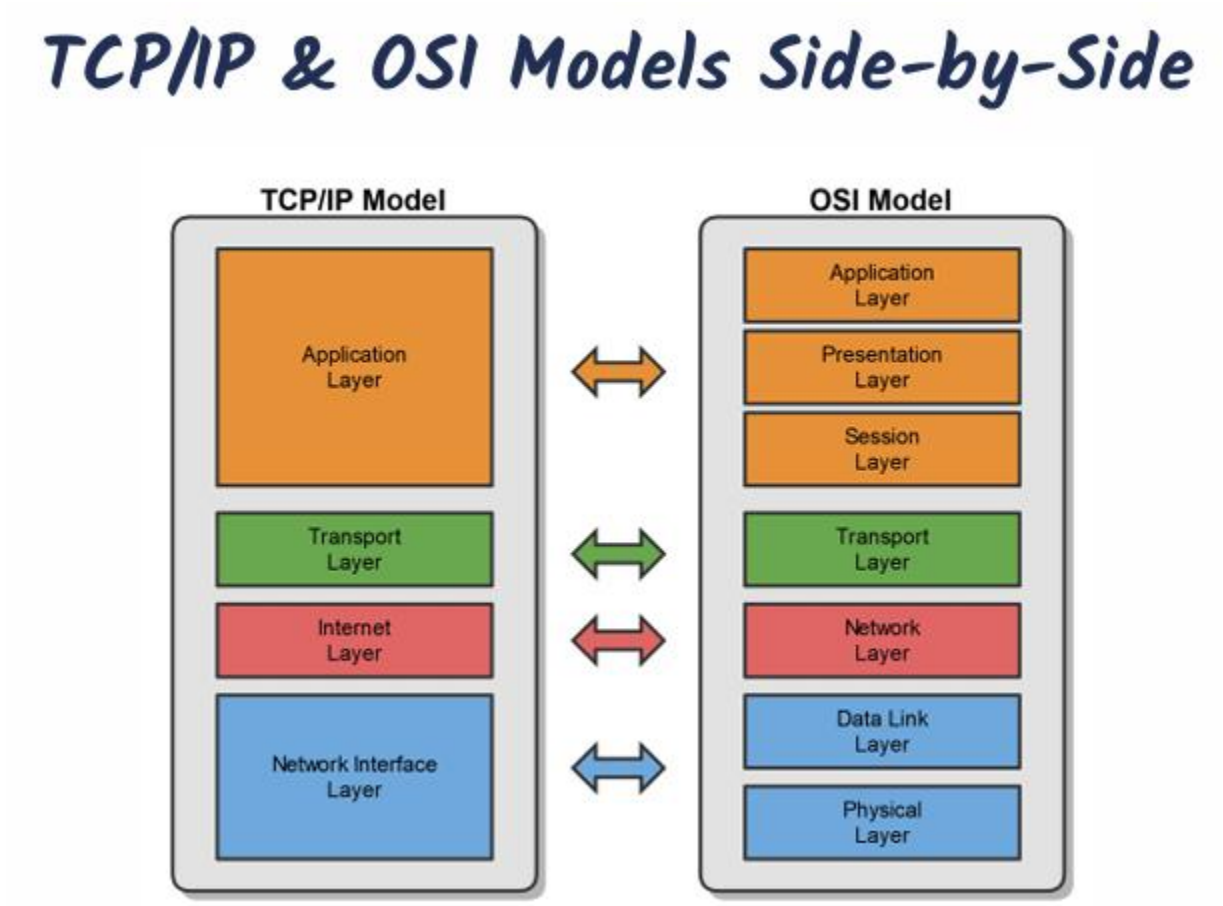
OSI Communication



The TCP/IP Model

- The TCP/IP suite is the most commonly used protocol suite in the networking world.
- It's essentially the protocol suite in which the Internet was built.
- It's the standard for computer networking.
- It is based on a 4-layer model that is similar to the OSI model.
- History of TCP/IP:
  - Developed by the United States Department of Defense (DoD) in the early 1970s.
  - In 1982, the DOD declared TCP/IP as the standard for all military computer networking.
  - In 1984, broad adoption of TCP/IP began (IBM, AT&T, etc.).





Overview of Network Security Protocols

Network security protocols are essential mechanisms used to **protect data in transit**, **authenticate devices and users**, and **safeguard remote and local network access**. Below are five fundamental protocols widely used in modern networks:

SSL/TLS (Secure Sockets Layer / Transport Layer Security)

Purpose:

> SSL (now largely replaced by TLS) is a protocol used to **secure data transmitted over a network** by encrypting it between two endpoints, such as a web browser and a web server.

How it works:

Uses a combination of **public key and symmetric key encryption** to secure data.

Ensures **confidentiality, integrity, and authentication**.

Importance:

Protects sensitive data like login credentials, credit card information, and personal messages during transmission.

Forms the basis for secure web browsing (`https` websites).

Use Cases:

Web traffic encryption.

Secure email (SMTPS).

Secure VPN tunnels.

IPsec (Internet Protocol Security)

Purpose:

A suite of protocols designed to **secure IP communications** by authenticating and encrypting each IP packet in a data stream.

How it works:

Operates at the **network layer**.

Uses protocols like **AH (Authentication Header)** and **ESP (Encapsulating Security Payload)** for securing packets.

Can work in **transport mode** (encrypts payload) or **tunnel mode** (encrypts entire packet).

Importance:

Provides end-to-end security in network communications.

Often used for **VPNs and securing site-to-site connections**.

Use Cases:

Site-to-site and remote-access VPNs.

Secure intra-organization communication.

SSH (Secure Shell)

Purpose:

Provides a **secure channel over an unsecured network** for services such as remote command-line login and remote command execution.

How it works:

Uses **public-key encryption and symmetric encryption** to protect data.

Requires **authentication through passwords, public/private key pairs, or certificates**.

Importance:

Replaces older, insecure protocols like Telnet.

Ensures **confidentiality and integrity** of data during remote administration.

Use Cases:

Secure remote server management.

Secure file transfers (via SCP or SFTP).

Tunneling other protocols securely.

VPN (Virtual Private Network)

Purpose:

Extends a **private network across a public network**, enabling users to send and receive data as if their devices were directly connected to the private network.

How it works:

Uses **encryption and tunneling protocols** (like IPsec, SSL/TLS, or PPTP) to secure data.

Creates a **virtual point-to-point connection** over public infrastructure.

Importance:

Protects sensitive data from eavesdropping on public networks.

Provides **secure remote access to corporate networks**.

Use Cases:

Remote employee access to office resources.

Bypassing geo-restrictions.

Securing public Wi-Fi connections.

802.1X (Port-Based Network Access Control)

Purpose:

A network access control protocol for **authenticating devices trying to connect to a LAN or WLAN**.

How it works:

Works at the **data link layer**.

Uses an **authentication server (like RADIUS)** to validate users/devices before granting access.

Importance:

Prevents unauthorized devices from accessing the network.

Supports **user/device-based authentication and dynamic VLAN assignment**.

Use Cases:

Enterprise Wi-Fi networks.

Secure wired switch ports.

Guest network segregation.

Why These Protocols Matter

These protocols collectively ensure that:

Sensitive data is encrypted during transit.

Unauthorized access is prevented through authentication.

Communications remain private and secure over public and private networks.

Organizations can securely **manage remote systems and users** without exposing internal resources.

Lab: Configuring SSL/TLS on a Web Server

Install Apache Web Server

Open your Linux VM terminal and run:

```
sudo apt update
sudo apt install apache2 -y
```

To confirm Apache is running:

Sudo systemctl enable apache2

Sudo systemctl start apache2

```
sudo systemctl status apache2
```

Visit `http://localhost` in your browser — you should see the Apache welcome page.

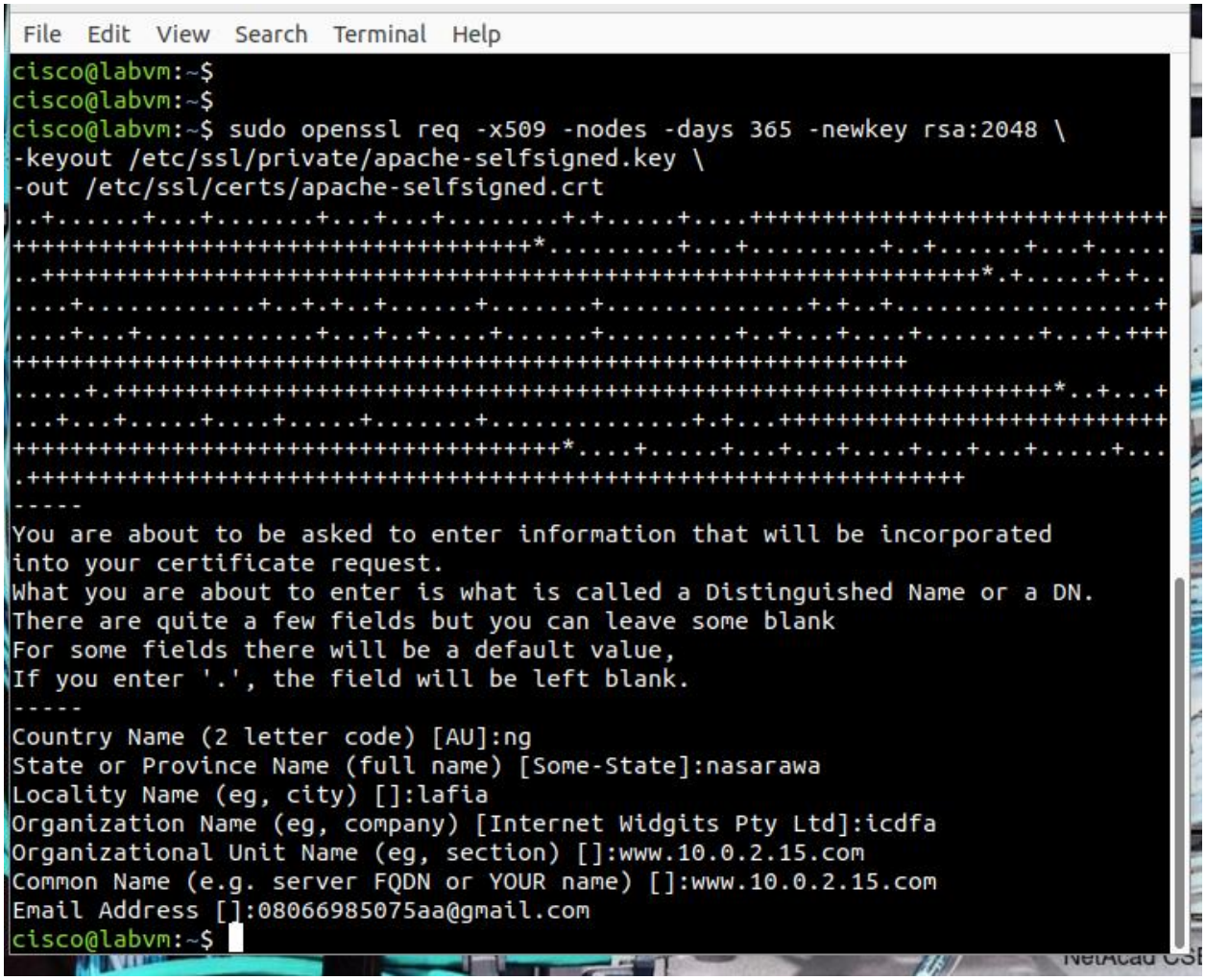
Obtain a Self-Signed SSL Certificate

Let's create a self-signed SSL certificate (valid for 365 days):

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/apache-selfsigned.key \
-out /etc/ssl/certs/apache-selfsigned.crt
```

You'll be prompted for:

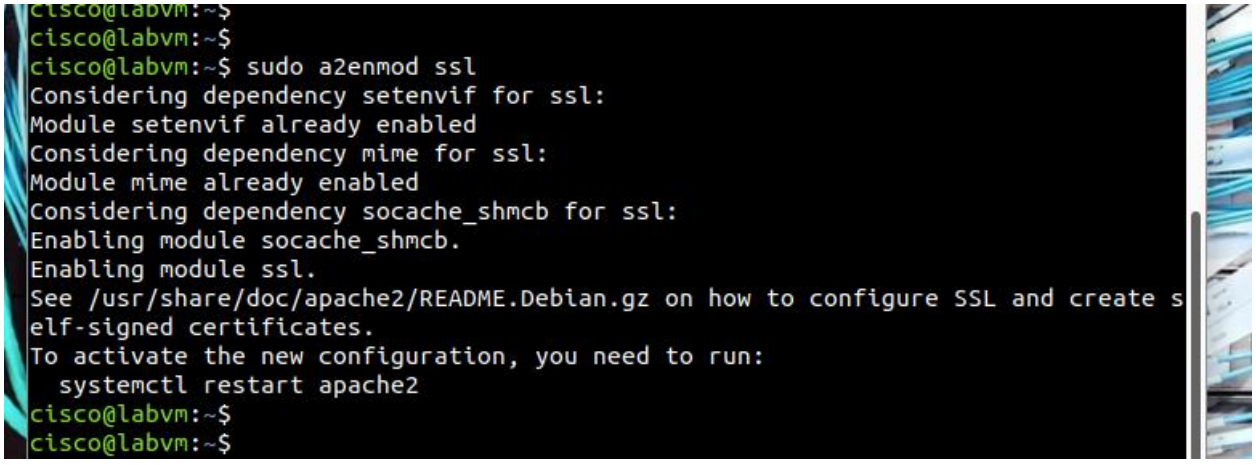
- Country
- State
- Organization
- Common Name (use your server's IP or domain)



Configure Apache for SSL/TLS

Enable SSL module

```
sudo a2enmod ssl
```



```
sudo systemctl restart apache2
```

Create an SSL Virtual Host file

Create a new config file:

```
sudo nano /etc/apache2/sites-available/www.10.0.2.15.com-ssl.conf
```

Paste the following configuration:

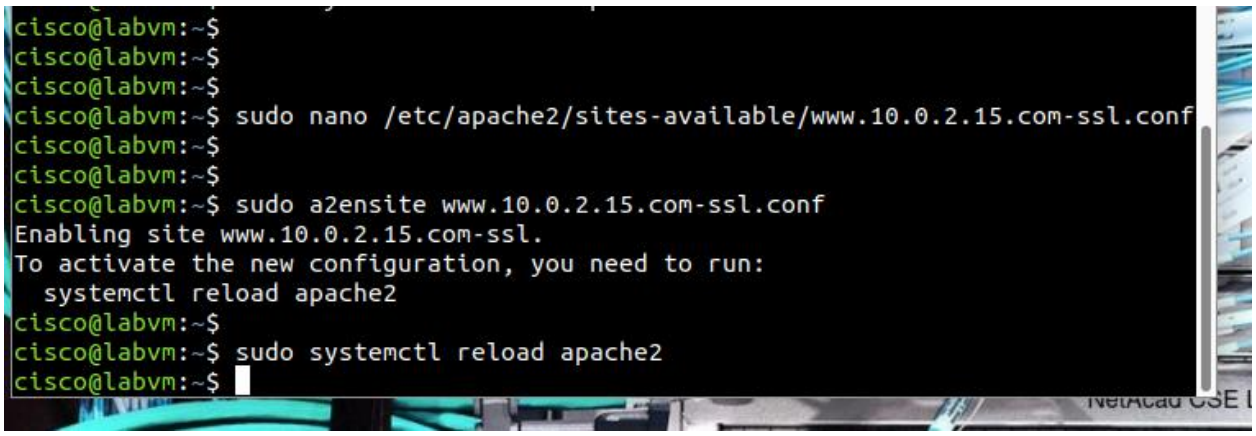
```
<VirtualHost *:443>
    ServerAdmin admin@www.10.0.2.15.com
    ServerName www.10.0.2.15.com
    DocumentRoot /var/www/html

    SSLEngine on
    SSLCertificateFile      /etc/ssl/certs/apache-selfsigned.crt
    SSLCertificateKeyFile   /etc/ssl/private/apache-selfsigned.key

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

Enable the SSL site

```
sudo a2ensite www.10.0.2.15.com-ssl.conf
sudo systemctl reload apache2
```



Redirect HTTP to HTTPS

Edit the default HTTP config:

```
sudo nano /etc/apache2/sites-available/000-default.conf
```

Add this line inside the `<VirtualHost *:80>` block:

```
Redirect "/" "https://10.0.2.15.com/"
```

Then reload Apache:

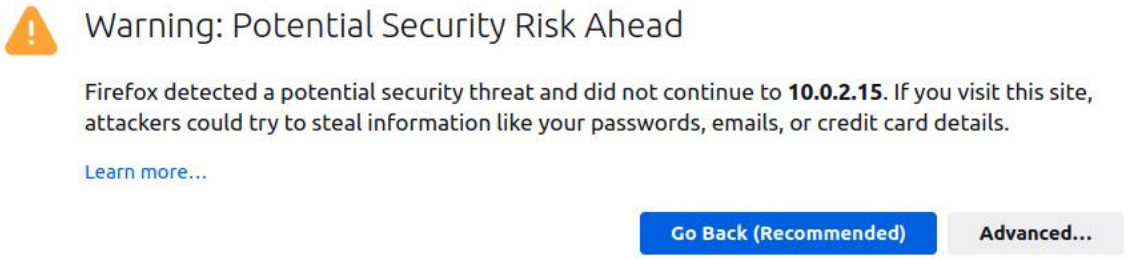
```
sudo systemctl reload apache2
```

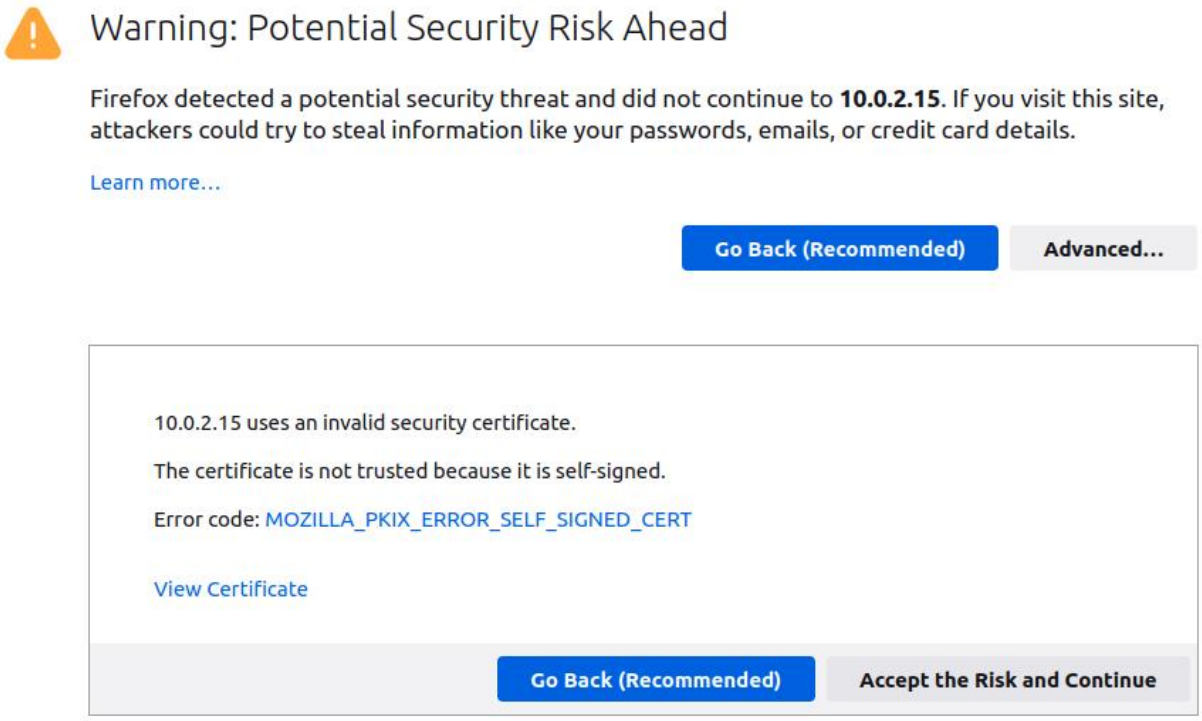
Test the Configuration

Open your browser.

Visit `https://<your-server-ip>` — you'll get a **security warning (self-signed certificate)**, proceed to view the site.







Success! Your Apache server now supports HTTPS with SSL/TLS.

Recap: What You Did

- Installed Apache.
- Generated a self-signed SSL certificate.
- Enabled SSL in Apache.
- Configured a Virtual Host for HTTPS.
- Redirected HTTP to HTTPS.
- Tested with a browser.

Lab: Implementing IPsec VPN using StrongSwan

VPN Server Setup (Ubuntu)

Install StrongSwan

```
sudo apt update
sudo apt install strongswan strongswan-pki libcharon-extra-plugins -y
```

Generate Certificates for VPN (using StrongSwan's PKI)

Create directories:

```
mkdir -p ~/pki/{cacerts,certs,private}
chmod 700 ~/pki
```

Generate CA certificate:

```
ipsec pki --gen --outform pem > ~/pki/private/ca-key.pem
ipsec pki --self --ca --lifetime 3650 --in ~/pki/private/ca-key.pem --type
rsa \
--dn "CN=VPN-CA" --outform pem > ~/pki/cacerts/ca-cert.pem
```

Generate server certificate:

```
ipsec pki --gen --outform pem > ~/pki/private/server-key.pem
ipsec pki --pub --in ~/pki/private/server-key.pem --type rsa | \
ipsec pki --issue --lifetime 1825 --cacert ~/pki/cacerts/ca-cert.pem --cakey
~/pki/private/ca-key.pem \
--dn "CN=vpnserver" --san "vpnserver" --flag serverAuth --outform pem >
~/pki/certs/server-cert.pem
```

Configure IPsec VPN

Copy certs to `/etc/ipsec.d`

```
sudo cp -r ~/pki/* /etc/ipsec.d/
```

Edit `/etc/ipsec.conf`

```
sudo nano /etc/ipsec.conf
```

Example config:

```
config setup
    charondebug="ike 2, knl 2, cfg 2, net 2, esp 2, dmn 2,  mgr 2"

conn vpn
    keyexchange=ikev2
    auto=add
    left=%any
    leftcert=server-cert.pem
    leftsubnet=0.0.0.0/0
    right=%any
    rightauth=eap-mschapv2
    rightdns=8.8.8.8
    rightsourceip=10.10.10.0/24
    eap_identity=%identity
```

Edit `/etc/ipsec.secrets`

```
sudo nano /etc/ipsec.secrets
```

Example content:

```
: RSA server-key.pem
testuser : EAP "testpassword"
```

Restart StrongSwan Service

```
sudo systemctl restart ipsec
```

VPN Client Setup (Linux)

Use **StrongSwan** or **ipsec.conf** on client side to match.

Monitor Encrypted Traffic with Wireshark

1 Start VPN connection from client

2 Open Wireshark and filter IPsec traffic:

```
esp || isakmp
```

This shows the **encrypted ESP (Encapsulating Security Payload)** packets and **ISAKMP negotiation**.

Confirm Encryption

Captured ESP traffic should appear **unreadable**, confirming encryption.

Recap:

- Installed StrongSwan
- Generated certificates
- Configured IPsec VPN server
- Connected a client
- Captured encrypted packets with Wireshark